

Mid-Atlantic Regression

Table of contents

1	Data	2
2	Angular Distance Between Observations	2
3	Euc Covariates	3
4	Our SvMF Model	3
5	Proposition 4 Submodels	3
6	Pretty Data Plot	4
7	Predictions	6
7.1	Table of Estimates	7
7.2	Plot	8
8	Angular Distance Between Predictions	9
9	LOOCV MSE	10
10	Residual Size	11
11	DoF	12
12	AIC	13
13	Main Figure	13
14	Appendix	14
14.1	Hessian of Likelihood at Optimum	14
14.2	Check Other Starts for vMF	14
14.3	Check Other Starts for SvMF	16

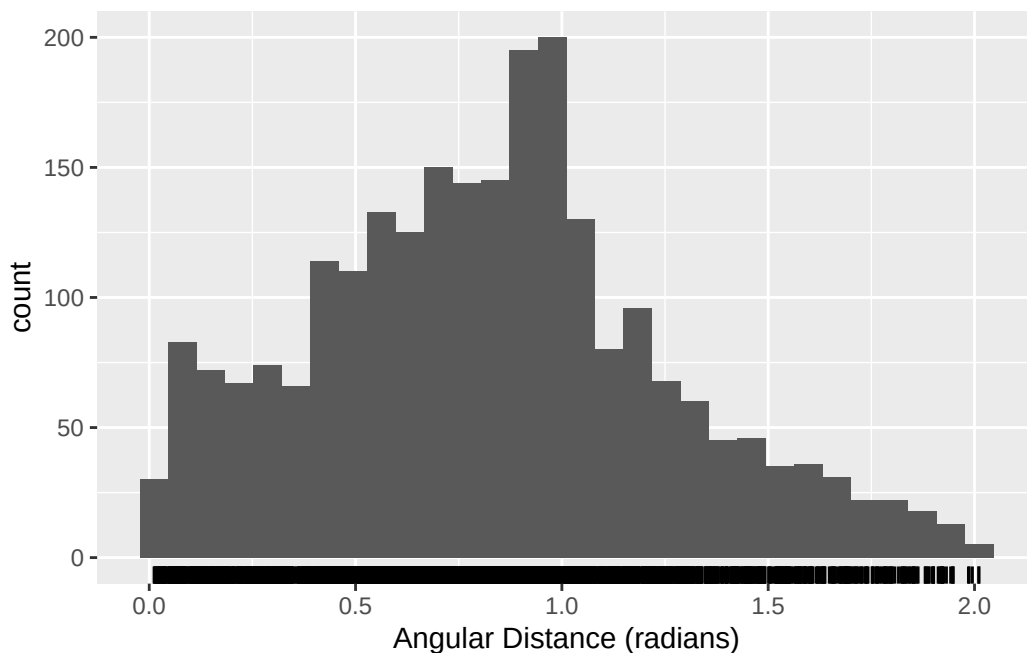
14.4 Likelihood Ratio Test of vMF vs SvMF	17
14.5 Session Information	22

Code version: 0.0.20

1 Data

2 Angular Distance Between Observations

```
Y <- fulldf %>%
  select(starts_with("Y")) %>%
  as.matrix()
ang_dist <- acos(pmin(Y %*% t(Y), 1)) # pmin clamps dot products to <=1, preventing NaN from
tibble::enframe(ang_dist[lower.tri(ang_dist)], value = "ang_dist") %>%
  ggplot() +
  geom_histogram(aes(x = ang_dist), bins = 30) +
  geom_rug(aes(x = ang_dist)) +
  scale_x_continuous(name = "Angular Distance (radians)")
```



3 Euc Covariates

```
xe <- fullrdf %>%  
  select(westedge)  
xestd <- xe %>% scale() %>% as_tibble()
```

4 Our SvMF Model

5 Proposition 4 Submodels

The four submodels discussed in the second paragraph of Section 5.1: the constrained links of Proposition 4(i) and 4(ii), each with and without the Euclidean covariate. Proposition 4(ii) fixes $B_s = I_{p-1}$, so the spherical part of the link is a plain rotation $\mu(x_s) = \widetilde{R}_0 x_s$. Proposition 4(i) relaxes that to an isotropic scale $B_s = \beta_s I_{p-1}$ with $0 < \beta_s \leq 1$, so $\beta_s = 1$ recovers Proposition 4(ii).

Proposition 4(i) is stated in the manuscript for $q_e = 0$. The + **Euclidean** variants below keep the restriction on the spherical component and add the same linear Euclidean term the **LinEuc** model uses. Unlike **mod** above, these are given the *unstandardised* covariate: the Proposition 4 fitters centre and scale it internally and store the transformation on the fitted link.

```
bind_rows(lapply(prop4mods, function(m) {  
  tibble(k = m$k, DoF = m$DoF, logLik = m$logLik, AIC = m$AIC)  
}), .id = "Model")
```

A tibble: 4 x 5

Model	k	DoF	logLik	AIC
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 Proposition 4(i): spherical only	66.3	11	103.	-184.
2 Proposition 4(i): spherical + all Euclidean	488.	15	235.	-440.
3 Proposition 4(ii): spherical only	62.2	8	98.8	-182.
4 Proposition 4(ii): spherical + all Euclidean	476.	14	233.	-439.

The isotropic spherical scale estimated by the Proposition 4(i) models. $\beta_s = 1$ corresponds to Proposition 4(ii), and $\phi = (1 - \beta_s)/(1 + \beta_s)$.

```
bind_rows(lapply(prop4mods[1:2], function(m) {  
  tibble(beta_s = m$mean$beta_s, phi = m$mean$phi,  
    psi_norm = sqrt(sum(m$mean$psi^2)))  
}), .id = "Model")
```

```
# A tibble: 2 x 4
  Model                                beta_s    phi psi_norm
  <chr>                                <dbl> <dbl>    <dbl>
1 Proposition 4(i): spherical only      0.872 0.0683    0.0683
2 Proposition 4(i): spherical + all Euclidean 0.929 0.0369    0.0369
```

The estimated scales a of the SvMF error distribution.

```
bind_rows(lapply(prop4mods, function(m) {
  as_tibble_row(setNames(as.list(m$a), paste0("a", seq_along(m$a))))
}), .id = "Model")
```

```
# A tibble: 4 x 4
  Model                                a1    a2    a3
  <chr>                                <dbl> <dbl> <dbl>
1 Proposition 4(i): spherical only      1  3.21 0.312
2 Proposition 4(i): spherical + all Euclidean 1  1.96 0.511
3 Proposition 4(ii): spherical only      1  3.17 0.316
4 Proposition 4(ii): spherical + all Euclidean 1  1.95 0.513
```

6 Pretty Data Plot

```
ymean <- colMeans(fulldf %>% select(starts_with("Y")))
ymean <- ymean/sqrt(sum(ymean^2))
#project to equator
ymeanproj <- c(ymean[1:2], 0)
ymeanproj/vnorm(ymeanproj)
```

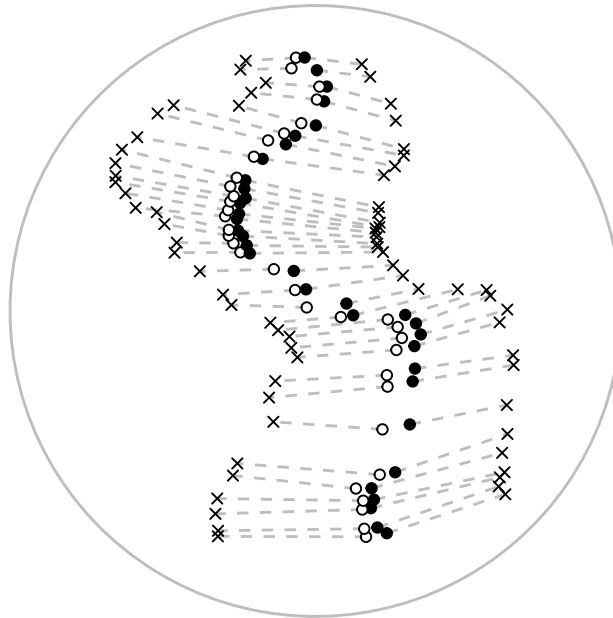
```
      Y1      Y2
0.8791608 -0.4765251  0.0000000
```

```
target <- c(1,0,0)
prettyrot <- parallel_transport_mat(ymeanproj, target)
prettyY <- (fulldf %>%
  select(Y1, Y2, Y3) %>%
  as.matrix()) %*% t(prettyrot)
colnames(prettyY) <- paste0("Y", 1:3)
prettyX <- (fulldf %>%
```

```

  select(X1, X2, X3) %>%
  as.matrix()) %*% t(prettyrot)
colnames(prettyX) <- paste0("X", 1:3)
prettyXmean <- colMeans(prettyX)
prettyXmean <- prettyXmean/vnorm(prettyXmean)
# denomloc <- drop(prettyrot %*% as_mobius_link_cann(mod$mean)$Qs[,1])
plotdata <- bind_cols(prettyY, prettyX, westedge = fulldf$westedge) %>%
  as_tibble() %>%
  ggplot() +
  geom_segment(aes(x=X2, y=X3, xend=Y2, yend=Y3), lty = "dashed", col = "grey") +
  geom_point(aes(x=Y2, y=Y3), shape = 4) +
  geom_point(aes(x=X2, y=X3, fill = westedge), show.legend = FALSE, shape = 21) +
  geom_path(data = circle_df, aes(x = x, y = y), inherit.aes = FALSE, color = "grey") +
  scale_fill_manual(values = c("black", "white")) +
  theme_void() +
  coord_fixed()
# mark mean of spherical covariate (obtained earlier)
# annotate("point", x = prettyXmean[2], y = prettyXmean[3], shape = 24, size = 2)
# annotate("point", x = denomloc[2], y = denomloc[3], shape = 3, size = 4, col = "blue")
plotdata

```



7 Predictions

The below ‘ours’ model is using LinEuc’s link with an extra covariate for an intercept and G01 free. The results here are from a local optimisation using gradient from default starting values. I’ve checked that global search for the best vMF regression does not find anything better.

```
cann <- as_mobius_link_cann(mod$mean)
cann
```

```
$P
      [,1]      [,2]      [,3]
Y1  0.4913098  0.5626852 -0.66483084
Y2 -0.4803629 -0.4616721 -0.74572810
Y3 -0.7265440  0.6857436  0.04346906
```

```
$Bs
      [,1]      [,2]
[1,] 0.9029649 0.0000000
[2,] 0.0000000 0.7702703
```

```
$Qs
      [,1]      [,2]      [,3]
X1  0.2657932  0.5868954 -0.7647927
X2 -0.5034121 -0.5920731 -0.6293058
X3 -0.8221498  0.5522710  0.1380812
```

```
$Be
      [,1]      [,2]
[1,] 0.07230452 0.0000000
[2,] 0.00000000 0.2525752
```

```
$Qe
      [,1]      [,2]      [,3]
dummyzero    1  0.0000000 0.0000000
westedge      0  0.1254316 0.9921023
ones          0 -0.9921023 0.1254316
```

```
$ce
[1] 1
```

```
attr("class")
[1] "mobius_link_cann" "list"
```

Lets try to interpret the fitted link.

```
cann$Bs %*% t(cann$Qs[,-1])
```

	X1	X2	X3
[1,]	0.5299459	-0.5346212	0.4986813
[2,]	-0.5890971	-0.4847356	0.1063599

```
cann$Be %*% t(cann$Qe[,-1])
```

	dummyzero	westedge	ones
[1,]	0	0.009069273	-0.07173348
[2,]	0	0.250580394	0.03168091

The first direction away from B_{01} (first row in above) is roughly equally influenced by X1, X2 and X3 (all are about 0.5) with west edge having very little influence (given the values of standardised westedge). The second direction away from B_{01} (second row in above) is roughly equally influenced by X1 and X2 but much less by X3 (which is the N-S direction) and westedge plays a role.

```
cann$Qs[,1]
```

	X1	X2	X3
	0.2657932	-0.5034121	-0.8221498

Some general scaling occurs with greater influence from X3 (N-S direction), then X2 then X1.

7.1 Table of Estimates

```
df <- cbind("diag($B_s$)" = diag(cann$Bs),
            "diag($B_e$)" = diag(cann$Be),
            tQe = t(cann$Qe[-1,-1])) %>%
  as.data.frame()
row.names(df) <- c("$t_2$", "$t_3$")
mykbl <- kbl(df, booktabs = TRUE, position = "!h", escape = FALSE, format = "latex", digits = 3,
  add_header_above(c(" " = 3, "$R_e^{\top}$" = 2), escape = FALSE)
mykbl
```

		$R_e^\top$		
	$\text{diag}(B_s)$	$\text{diag}(B_e)$	westedge	ones
t_2	0.90	0.07	0.13	-0.99
t_3	0.77	0.25	0.99	0.13

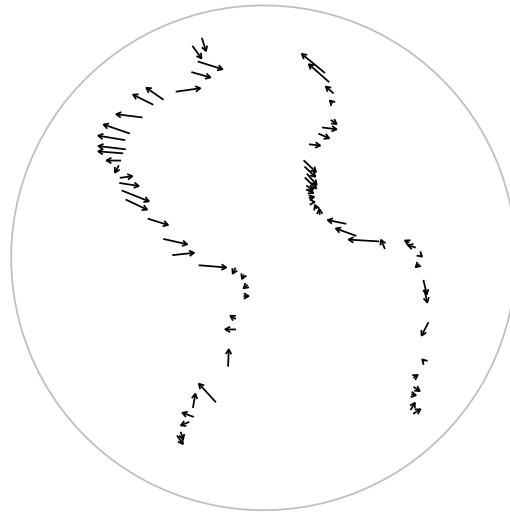
7.2 Plot

Note that we have not optimised Q for ESAG2 because optimisation did not converge.

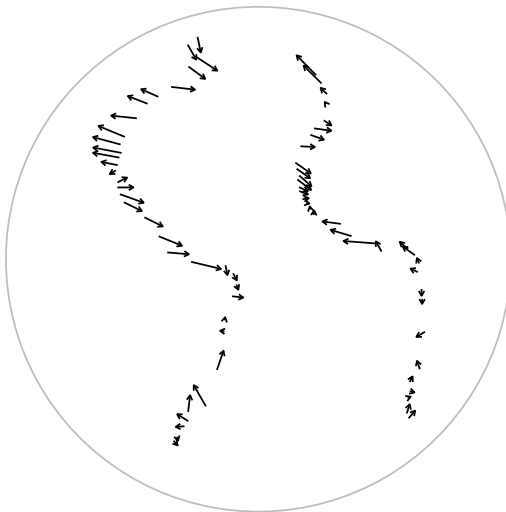
Ours



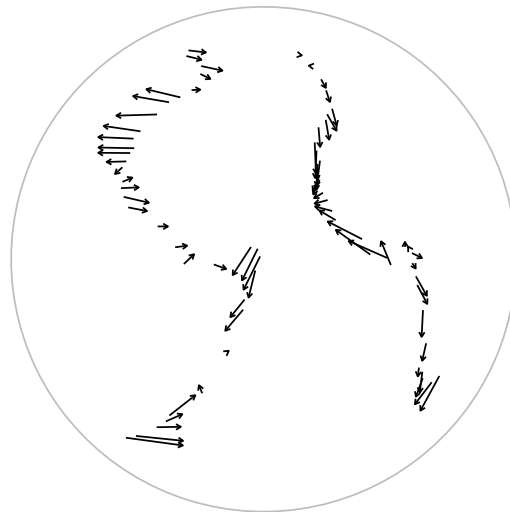
IAG1



ESAG1



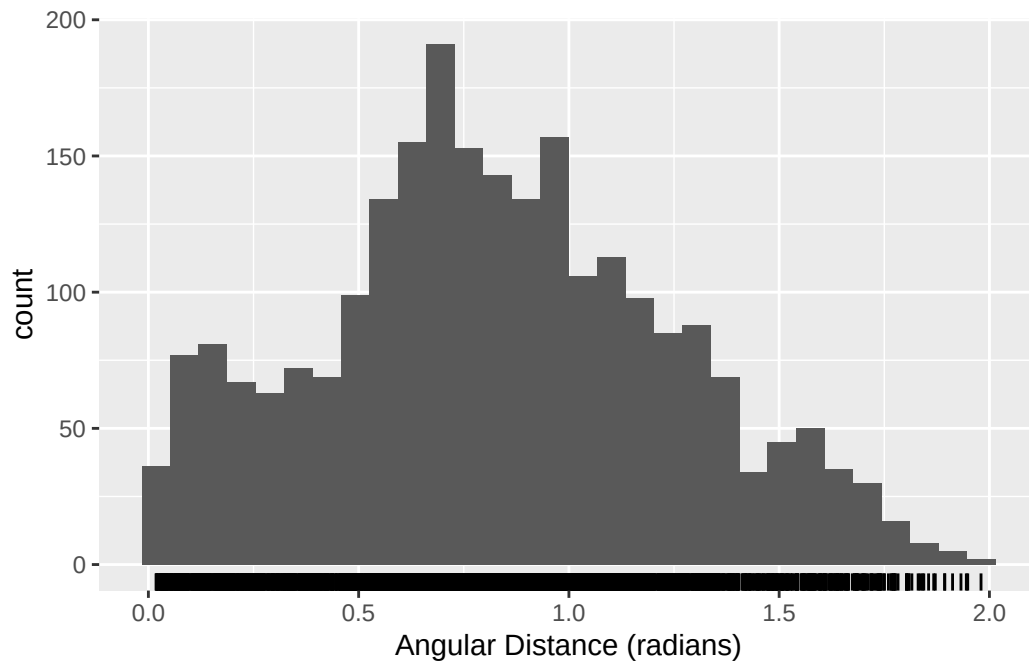
ESAG2



8 Angular Distance Between Predictions

```
pred_ang_dist <- acos(pmin(mod$pred %*% t(mod$pred), 1))
tibble::enframe(pred_ang_dist[lower.tri(pred_ang_dist)], value = "ang_dist") %>%
  ggplot() +
  geom_histogram(aes(x = ang_dist), bins = 30) +
```

```
geom_rug(aes(x = ang_dist)) +
scale_x_continuous(name = "Angular Distance (radians)")
```



9 LOOCV MSE

```
loocvmseSvMF <- function(mod){
  stopifnot(inherits(mod$mean, "mobius_link_Omega"))
  dists <- pbapply::pblapply(1:nrow(mod$y), function(idx){
    newmod <- mobius_SvMF(mod$y[-idx,],
      xs = mod$xs[-idx,],
      xe = mod$xe[-idx,c(-1,-ncol(mod$xe)), drop = FALSE],
      fix_qs1 = FALSE,
      type = "LinEuc",
      G0behaviour = "free",
      mean = mod$mean,
      k = mod$k,
      a = mod$a,
      G0 = mod$G0)
    pred <- mobius_link(xs = mod$xs[idx,, drop = FALSE],
      xe = mod$xe[idx,, drop = FALSE],
```

```

      param = newmod$mean)
  obs <- mod$y[idx,]
  Euc <- vnorm(drop(obs - pred))
  angle <- acos(rowSums(obs * pred))
  return(c(
    Euc = Euc,
    angle = angle
  ))
})
dists <- dists %>%
  simplify2array() %>%
  t() %>%
  as_tibble()
dists %>%
  summarise(across(everything(), ~sum(.x^2)/nrow(mod$y)))
}
loocvmseSvMF(mod)

```

```

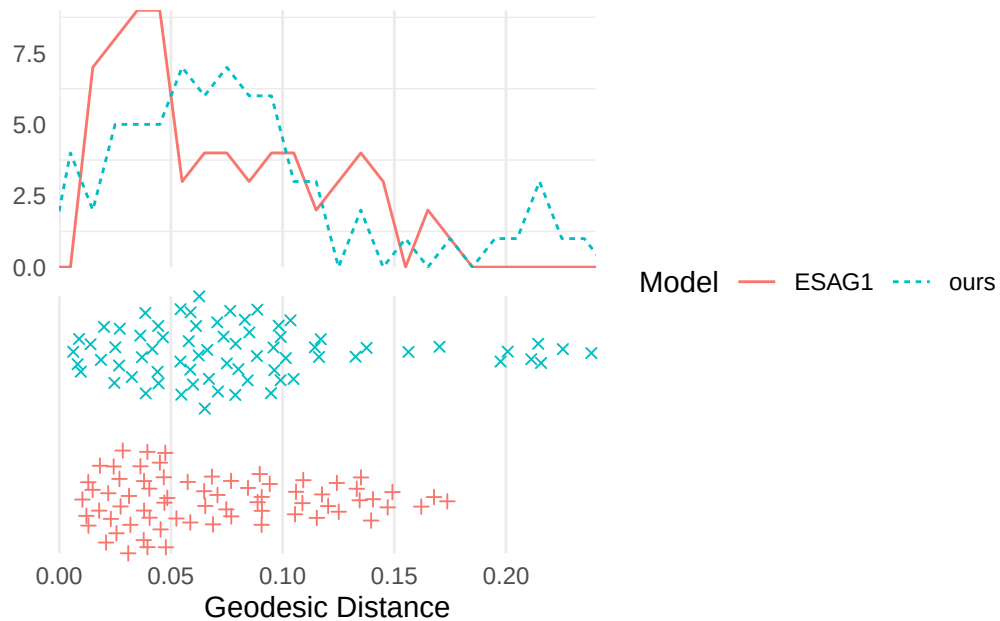
# A tibble: 1 x 2
  Euc   angle
  <dbl> <dbl>
1 0.0107 0.0107

```

Both of these are smaller than the LOOCV MSE that Rosenthal's PLT achieved of 0.074, which corresponds to the Euc metric MSE.

10 Residual Size

Geodesic distance between predicted mean and observation



11 DoF

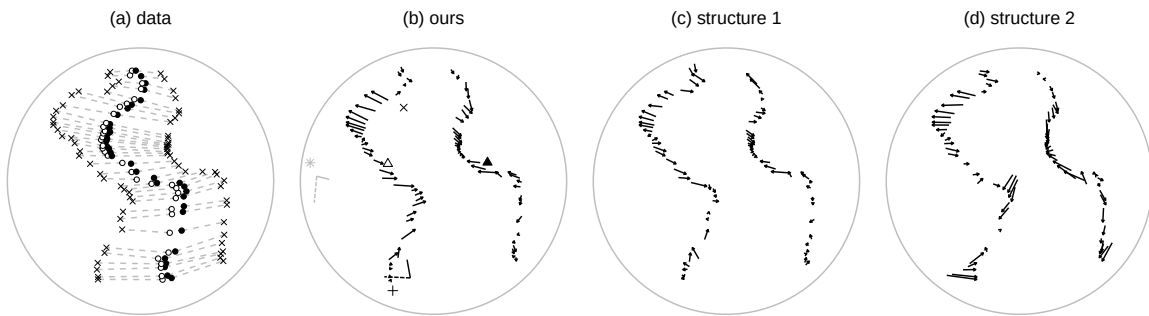
Ours	16
IAG1	17
ESAG1	22
ESAG2	25
Proposition 4(i): spherical only	11
Proposition 4(i): spherical + all Euclidean	15
Proposition 4(ii): spherical only	8
Proposition 4(ii): spherical + all Euclidean	14

12 AIC

	Ours
	-463.2407
	IAG1
	-370.7254
	ESAG1
	-459.5946
	ESAG2
	-436.2852
Proposition 4(i): spherical only	
	-184.0309
Proposition 4(i): spherical + all Euclidean	
	-439.9135
Proposition 4(ii): spherical only	
	-181.5131
Proposition 4(ii): spherical + all Euclidean	
	-438.5591

13 Main Figure

```
(plotdata + ggtitle("(a) data") + theme(plot.title = element_text(hjust = 0.5))) +  
(plotours + theme(legend.position = "none", plot.title = element_text(hjust = 0.5)) + ggtitle("  
  (plotESAG1 + ggtitle("(c) structure 1") + theme(plot.title = element_text(hjust = 0.5))) +  
  (plotESAG2 + ggtitle("(d) structure 2") + theme(plot.title = element_text(hjust = 0.5))) +  
  plot_layout(ncol = 4, widths = c(3,3,3,3))
```



```
ggsave("midatlantic_fig.pdf", width = 12, height = 3.5)
```

Caption: Regression for the midatlantic ridge data. From left: midatlantic ridge (circles) and corresponding locations on the continent (crosses) from Rosenthal et al (2014); our regression; Paine et al structure 1 regression; Paine et al structure 2 regression. The sphere is shown orthogonally projected with north pointing up the page. Arrows: start at the predicted mean and end at the observed continental location, and thus represent residuals. Filled symbols: eastern side. Unfilled symbols: western side. Triangles: Mean of ridge locations and corresponding predicted mean for western or eastern side. Plus symbol: estimated value of r_{s1} . Pair of black lines: direction of the estimated second (solid) and third (dashed) columns of B_0 located at the estimated first column b_{01} of B_0 . Pair of grey lines: estimated directions of γ_{02} (solid) and γ_{03} (dashed) located at the estimated γ_{01} .

14 Appendix

14.1 Hessian of Likelihood at Optimum

The parameter vector is longer than the DoF because of the constraints in the optimisation. There should be $\text{DoF} + (3-1) * (3 - 2) / 2$ positive eigenvalues. The term $(3-1) * (3 - 2) / 2$ is for the commutativity constraint on Omega, which the likelihood computation does not account for, so appears as extra degrees of freedom in the Hessian of the likelihood.

```
[1] 8.254097e+03 6.672960e+03 2.990692e+03 2.470445e+03 1.236501e+03
[6] 3.582921e+02 9.508065e+01 2.869573e+01 2.125180e+01 9.630585e+00
[11] 5.113148e+00 4.746750e+00 2.960024e+00 1.951304e+00 1.054190e+00
[16] 4.074145e-01 2.936479e-06
```

These are all positive and non-zero, which confirms that the optimisation routine has found a local maximum of the likelihood.

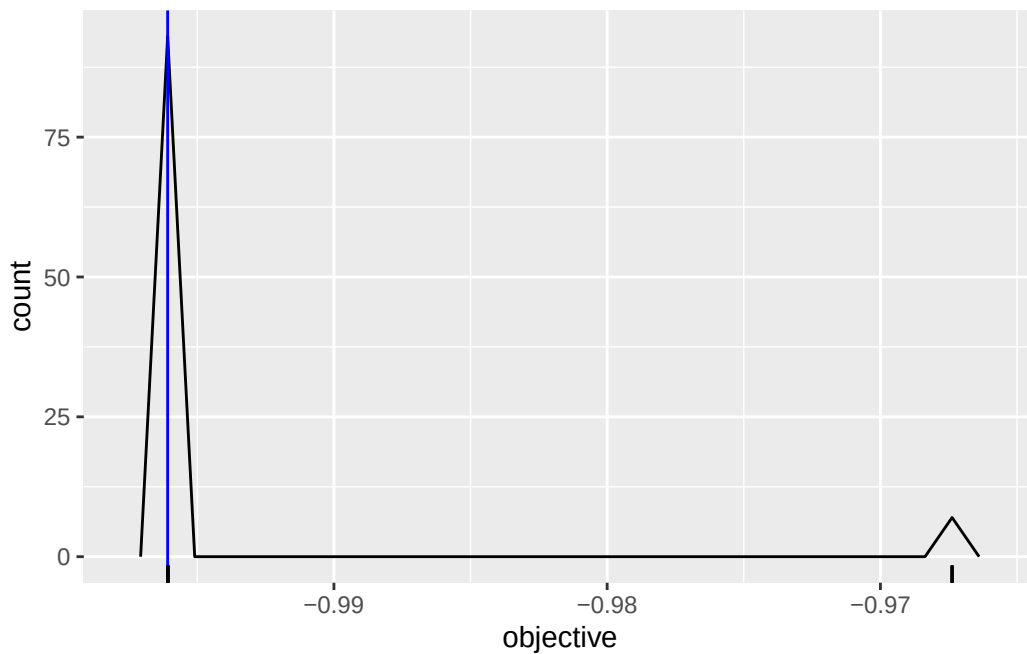
14.2 Check Other Starts for vMF

```
restarts <- pbapply::pblapply(1:100, function(seed){
  start <- rand_mobius_link_cann(p = 3, qs = 3, qe = ncol(xe) + 2, preseed = seed)
  # convert to LinEuc form:
  set.seed(seed+1)
  Qe <- mclust::randomOrthogonalMatrix(ncol(xe)+1, 3-1)
  bigQe <- cbind(0, rbind(0, Qe))
```

```

bigQe[, 1] <- 0
bigQe[1,1] <- 1
start$Qe <- bigQe
start$ce <- 1
modvMF <- mobius_vMF(y = fulldf %>% select(starts_with("Y")) %>% as.matrix(),
  xs = fulldf %>% select(starts_with("X")) %>% as.matrix(),
  xe = xe %>% as.matrix(),
  fix_qs1 = FALSE, type = "LinEuc",
  start = start)
}, cl = 2)
lapply(restarts, "[", "obj") %>%
  unlist() %>%
  enframe("seed", "objective") %>%
  ggplot()+
  geom_freqpoly(aes(x = objective), bins = 30) +
  geom_vline(xintercept = mod$preest$nlopt$objective, col = "blue") +
  geom_rug(aes(x = objective))

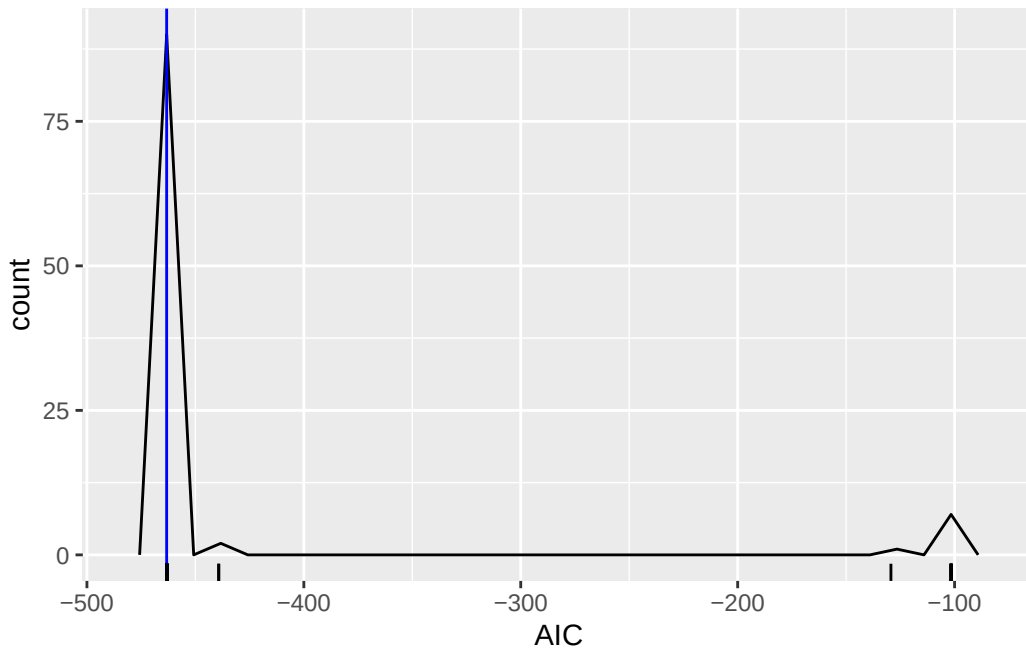
```



Other initial parameters have not improved on the default initial parameters.

14.3 Check Other Starts for SvMF

```
restarts <- pbapply::pblapply(1:100, function(seed){
  # randomly generates a SpEuc-form link
  start <- rand_mobius_link_cann(p = 3, qs = 3, qe = ncol(xestd) + 2, preseed = seed)
  # convert to LinEuc form:
  set.seed(seed+1)
  Qe <- mclust::randomOrthogonalMatrix(ncol(xestd)+1, 5-1)
  bigQe <- cbind(0, rbind(0, Qe))
  bigQe[, 1] <- 0
  bigQe[1,1] <- 1
  start$Qe <- bigQe
  start$ce <- 1
  mobius_SvMF(y = fulldf %>% select(starts_with("Y")) %>% as.matrix(),
              xs = fulldf %>% select(starts_with("X")) %>% as.matrix(),
              xe = xestd %>% as.matrix(),
              type = "LinEuc",
              G01behaviour = "free",
              mean = start)
}, cl = 4)
badrestarts <- unlist(lapply(restarts, inherits, "try-error"))
restarts <- restarts[!badrestarts]
lapply(restarts, "[", "AIC") %>%
  unlist() %>%
  tibble::enframe("seed", "AIC") %>%
  ggplot()+
  geom_freqpoly(aes(x = AIC), bins = 30) +
  geom_vline(xintercept = mod$AIC, col = "blue") +
  geom_rug(aes(x = AIC))
```



Other initial parameters have not improved on the default initial parameters.

14.4 Likelihood Ratio Test of vMF vs SvMF

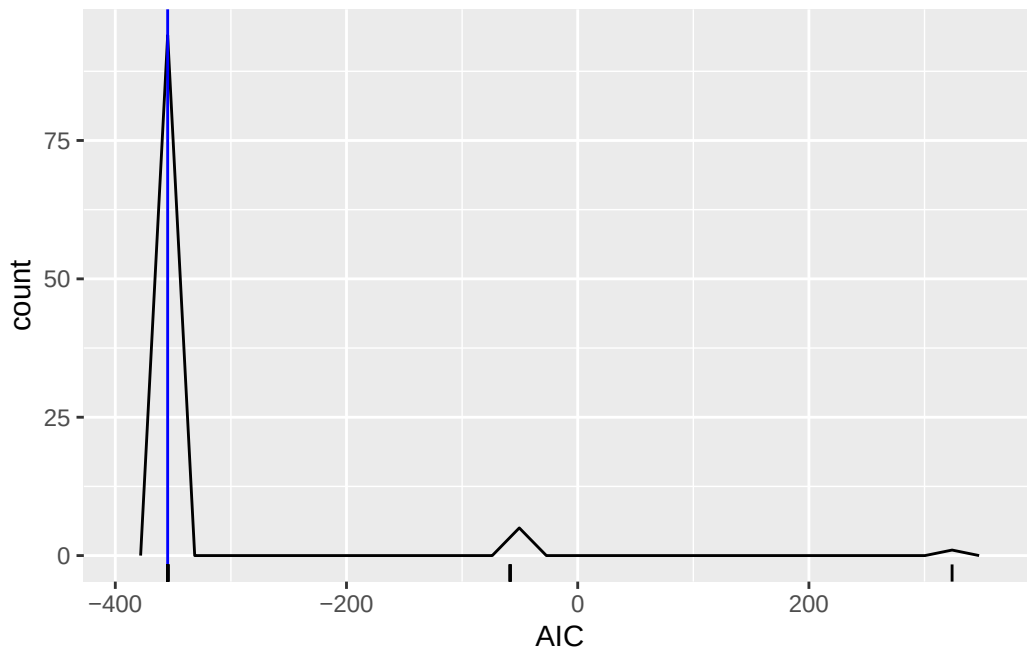
First get best vMF model via a searching with 100 restarts.

```
mod_vMF <- mobius_vMF(y = mod$y,
                     xs = mod$xs,
                     xe = mod$xe,
                     fix_qs1 = FALSE,
                     type = "LinEuc")
mod_vMF_restarts <- pbapply::pblapply(1:100, function(seed){mobius_vMF_refit(mod_vMF, seed)})
```

Warning in mobius_vMF_general(y = y, xs = xs, xe = xe, start = start, type = type, : NLOPT_MAXEVAL_REACHED: Optimization stopped because maxeval (above) was reached.

Error in mobius_link_Omega_check(projectedom) :
The following checks failed. Omega_comm: 0.0043

```
lapply(mod_vMF_restarts, "[", "AIC") %>%
  unlist() %>%
  tibble::enframe("seed", "AIC") %>%
  ggplot()+
  geom_freqpoly(aes(x = AIC), bins = 30) +
  geom_vline(xintercept = mod_vMF$AIC, col = "blue") +
  geom_rug(aes(x = AIC))
```



```
idx <- which.min(lapply(mod_vMF_restarts, "[", "AIC") %>% unlist())
mod_vMF <- mod_vMF_restarts[[idx]]
mod_vMF$k
```

```
[1] 255.3967
```

```
mod_vMF$AIC
```

```
[1] -354.6918
```

Below is function for comparing likelihoods for responses y . The scaled von Mises Fisher regression uses `mod_vMF` for initial parameters of concentration and the mean link. We use the default initial parameters for `a` and `G0` because under the null these aren't identifiable, and so on simulated data they are unstable.

```

getlikR <- function(y){
  mod1 <- mobius_vMF(y,
    xs = mod_vMF$xs,
    xe = mod_vMF$xe,
    type = "LinEuc",
    fix_qs1 = FALSE,
    start = mod_vMF$est)
  mod2 <- mobius_SvMF(y,
    xs = mod_vMF$xs,
    xe = mod_vMF$xe,
    type = "LinEuc",
    fix_qs1 = FALSE,
    G0behaviour = "free",
    mean = mod_vMF$mean,
    k = mod_vMF$k)
  if ((mod1$nlOPT$status != 4) || (mod2$nlOPT$status != 4)){
    return(list(likR = NA_real_,
      vMF = mod1,
      SvMF = mod2))
  }
  lLik1 <- mobius_SvMF_log_lik(mod1$y, xs = mod1$xs, xe = mod1$xe,
    mean = mod1$est,
    k = mod1$k,
    a = rep(1, 3),
    G0 = mod2$G0) %>%
    colSums()
  lLik2 <- mobius_SvMF_log_lik(mod2$y, xs = mod2$xs, xe = mod2$xe,
    mean = mod2$mean,
    k = mod2$k,
    a = mod2$a,
    G0 = mod2$G0) %>%
    colSums()

  list(likR = -2* (lLik1[["R"]] - lLik2[["R"]]),
    vMF = mod1,
    SvMF = mod2)
}

```

Now we simulate the response 1000 times from the best vMF model and compare the likelihood of the vMF model to the likelihood of the SvMF model on each simulated data.

```

null_likRs <- pbapply::pblapply(1:1000, function(seed){
  set.seed(seed)
  y_ld <- rmobius_SvMF(xs = mod$xs,
                      xe = mod_vMF$xe,
                      mean = mod_vMF$mean,
                      k = mod_vMF$k,
                      a = rep(1, 3),
                      G0 = diag(1, 3))

  y <- y_ld[, -4]
  getlikR(y)
}, cl = 3)

obs <- getlikR(mod$y)

```

```

null_likRs_vec <- lapply(null_likRs, "[", "likR") %>% unlist()

sum(is.na(null_likRs_vec))

```

```
[1] 3
```

```
sum(!is.na(null_likRs_vec))
```

```
[1] 997
```

Three didn't converge before the default number of iterations. Suspect the initial G0 was a long way from the optimum G0 and with the Cayley transform iterations are getting less effective. Restart using current G0 estimate as the reference coordinates for G0.

```

finished_likRs <- lapply(null_likRs[is.na(null_likRs_vec)], function(x){x$SvMF
  mod2 <- mobius_SvMF(x$SvMF$y,
                      xs = x$SvMF$xs,
                      xe = x$SvMF$xe,
                      type = "LinEuc",
                      fix_qs1 = FALSE,
                      G0behaviour = "free",
                      mean = x$SvMF$mean,
                      k = x$SvMF$k,
                      G0 = x$SvMF$G0,
                      a = x$SvMF$a)

  llik2 <- mobius_SvMF_log_lik(mod2$y, xs = mod2$xs, xe = mod2$xe,

```

```

      mean = mod2$mean,
      k = mod2$k,
      a = mod2$a,
      G0 = mod2$G0) %>%
  colSums()
lLik1 <- mobius_SvMF_log_lik(x$vMF$y, xs = x$vMF$xs, xe = x$vMF$xe,
  mean = x$vMF$est,
  k = x$vMF$k,
  a = rep(1, 3),
  G0 = mod2$G0) %>%
  colSums()
list(likR = -2* (lLik1[["R"]] - lLik2[["R"]]),
  vMF = x$vMF,
  SvMF = mod2)
})
lapply(finished_likRs, function(x) x$SvMF$nlopt$status) |> unlist()

```

```
[1] 4 4 4
```

They all converge very quickly with good reference coordinates for G0.

```

null_likRs[is.na(null_likRs_vec)] <- finished_likRs
null_likRs_vec <- lapply(null_likRs, "[", "likR") %>% unlist()

```

```
obs$likR
```

```
[1] 92.53948
```

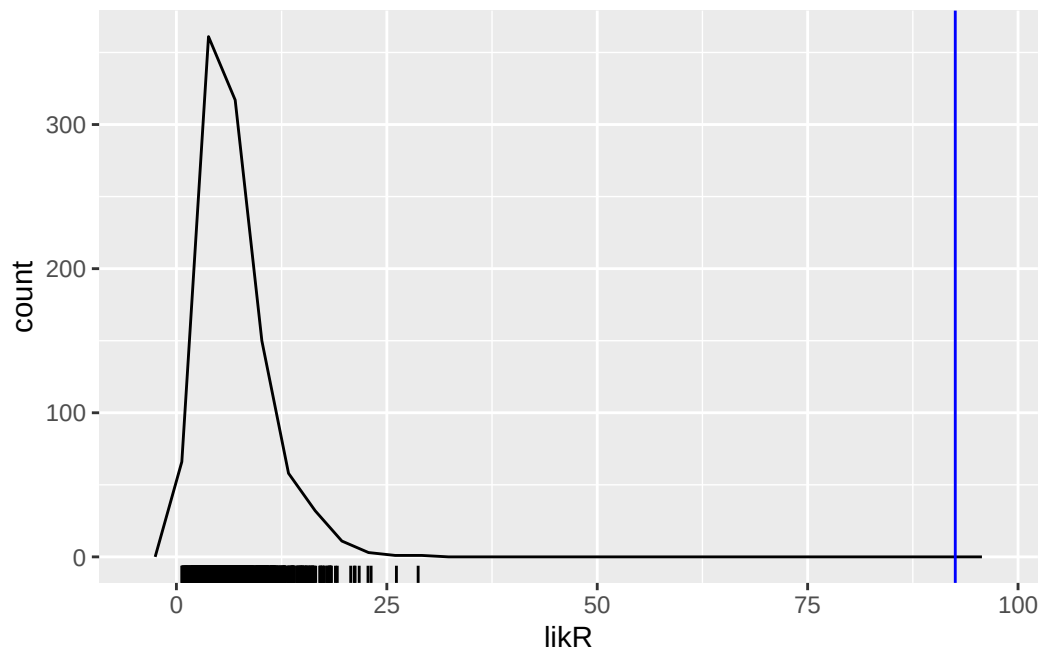
```
range(null_likRs_vec, na.rm = TRUE)
```

```
[1] 0.6434721 28.7225790
```

```

tibble::enframe(null_likRs_vec, "seed", "likR") %>%
  ggplot() +
  geom_freqpoly(aes(x = likR), bins = 30) +
  geom_vline(xintercept = obs$likR, col = "blue") +
  geom_rug(aes(x = likR))

```



p-value is the probability, under the null, of getting a likelihood-ratio that is at least as large as the observed ratio:

```
mean(null_likRs_vec > obs$likR, na.rm = TRUE)
```

```
[1] 0
```

The observed likelihood ratio is far larger than any value obtained under the null, so this small p -value suggests that the data was not drawn from a vMF regression.

14.5 Session Information

```
sessionInfo()
```

```
R version 4.3.3 (2024-02-29)
Platform: x86_64-pc-linux-gnu (64-bit)
Running under: Ubuntu 24.04.4 LTS
```

```
Matrix products: default
BLAS: /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
```

LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0

locale:

```
[1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C
[3] LC_TIME=en_US.UTF-8      LC_COLLATE=en_US.UTF-8
[5] LC_MONETARY=en_US.UTF-8  LC_MESSAGES=en_US.UTF-8
[7] LC_PAPER=en_US.UTF-8     LC_NAME=C
[9] LC_ADDRESS=C             LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
```

time zone: Australia/Sydney

tzcode source: system (glibc)

attached base packages:

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

other attached packages:

```
[1] kableExtra_1.4.1 patchwork_1.3.2  dplyr_1.2.1      tibble_3.3.1
[5] sphm_0.0.20      ggplot2_4.0.3    R.matlab_3.7.0
```

loaded via a namespace (and not attached):

```
[1] tidyr_1.3.2          utf8_1.2.6          generics_0.1.4
[4] xml2_1.6.0           stringi_1.8.9       hms_1.1.4
[7] digest_0.6.39        magrittr_2.0.4      RcppEigen_0.3.4.0.2
[10] evaluate_1.0.5       grid_4.3.3          RColorBrewer_1.1-3
[13] fastmap_1.2.0        R.oo_1.27.1         jsonlite_2.0.0
[16] R.utils_2.13.0       progress_1.2.3      scorematchingad_0.1.6
[19] mclust_6.1.3         purrr_1.2.2         viridisLite_0.4.3
[22] scales_1.4.0         pbapply_1.7-4       codetools_0.2-19
[25] textshaping_1.0.5    Rdpack_2.6.6        cli_3.6.5
[28] crayon_1.5.3         rlang_1.1.7         rbibutils_2.4.1
[31] R.methodsS3_1.8.2    withr_3.0.2         yaml_2.3.12
[34] otel_0.2.0           ggbeeswarm_0.7.2    parallel_4.3.3
[37] tools_4.3.3          nloptr_2.2.1        vctrs_0.7.2
[40] R6_2.6.1             lifecycle_1.0.5     stringr_1.6.0
[43] vipor_0.4.7          ragg_1.2.7          beeswarm_0.4.0
[46] pkgconfig_2.0.3      pillar_1.11.1       gtable_0.3.6
[49] glue_1.8.0           Rcpp_1.1.2          systemfonts_1.3.2
[52] xfun_0.60            tidyselect_1.2.1    rstudioapi_0.19.0
[55] knitr_1.51           dichromat_2.0-0.1   farver_2.1.2
[58] htmltools_0.5.9      rmarkdown_2.31      svglite_2.2.2
[61] labeling_0.4.3       compiler_4.3.3      prettyunits_1.2.0
[64] S7_0.2.2
```